

Housing Platform API Docs

October 8, 2026

CONTENTS

Housing & Roommate Management Platform — API Docs	3
Conventions	3
1. Auth — /auth	4
2. Users — /users	4
3. Subscriptions — /subscriptions	5
4. Properties — /properties	5
5. Rooms — /properties/:propertyId/rooms + /rooms	6
6. Roommate Matching — /roommates	6
7. Viewing Requests — /viewing-requests	7
8. Applications — /applications	7
9. Leases — /leases	8
10. Rent Payments — /rent-payments	8
11. Utility Bills — /utility-bills	8
12. Maintenance — /maintenance	9
13. Documents — /documents	9
14. Notifications — /notifications	9
15. Payments (SSLCommerz) — /payments	10
16. Dashboard — /dashboard	10
17. Assistant — /assistant (not in Postman collection)	10
Quick test flow	11

Housing & Roommate Management Platform — API Docs

Base URL (local): `http://localhost:5000` API prefix: `http://localhost:5000/api/v1`

Health check: `GET /`

Generated from `backend/postman_collection.json` + `src/modules/*/*.route.ts`.

Postman variable `{{baseUrl}}` = `http://localhost:5000/api/v1`.

Conventions

- Every response uses the envelope `{ success, message, data | errors }`.
- Auth: stateless JWT. Send `Authorization: Bearer <accessToken>`.
- Login/register return `{ user, accessToken, refreshToken }`.
- Refresh: `POST /auth/refresh` with `{ refreshToken }`.
- Roles: `ADMIN`, `OWNER`, `MANAGER`, `TENANT`. Owner/manager scope is enforced per-property in services.
- Rate limits: global 300/15 min; register/login/forgot/reset 8/15 min; refresh/google 20/15 min.
- Errors: `ApiError` + Prisma `P2002` / `P2025` mapped to envelope shape. `forgot-password` always returns the same message.

1. Auth — /auth

Method	Endpoint	Auth	Notes
POST	/auth/register	public	{ name, email, password }
POST	/auth/login	public	{ email, password }
POST	/auth/google	public	{ idToken } Google OAuth
POST	/auth/refresh	public	{ refreshToken }
POST	/auth/logout	Bearer	clears stored refresh token
GET	/auth/me	Bearer	current user
POST	/auth/forgot-password	public	{ email }, same response either way, 1 h hashed token
POST	/auth/reset-password	public	{ token, newPassword }
POST	/auth/verify-email	public	{ token }
POST	/auth/resend-verification	public	{ email }

2. Users — /users

Method	Endpoint	Auth
GET	/users/me/profile	Bearer
PATCH	/users/me	Bearer
GET	/users	ADMIN
PATCH	/users/:id/ban	ADMIN
PATCH	/users/:id/verify	ADMIN
PATCH	/users/:id/role	ADMIN

3. Subscriptions — `/subscriptions`

Method	Endpoint	Auth
POST	<code>/subscriptions/checkout</code>	OWNER/MANAGER
GET	<code>/subscriptions/my</code>	Bearer
GET	<code>/subscriptions?status=PENDING</code>	ADMIN
PATCH	<code>/subscriptions/:id/approve</code>	ADMIN

4. Properties — `/properties`

Method	Endpoint	Auth
GET	<code>/properties/my</code>	OWNER/MANAGER scoped list
POST	<code>/properties</code>	OWNER
GET	<code>/properties?city=Dhaka</code>	public, cached 30 s (Redis optional)
GET	<code>/properties/:propertyId</code>	public
PATCH	<code>/properties/:propertyId</code>	OWNER/ADMIN
DELETE	<code>/properties/:propertyId</code>	OWNER/ADMIN soft delete
POST	<code>/properties/:propertyId/images</code>	OWNER multipart images to Cloudinary
GET	<code>/properties/:propertyId/tenants</code>	OWNER active tenants from leases
PATCH	<code>/properties/:propertyId/manager</code>	OWNER assign by email, auto-promotes TENANT
DELETE	<code>/properties/:propertyId/manager</code>	OWNER unassign

5. Rooms — `/properties/:propertyId/rooms` + `/rooms`

Method	Endpoint	Auth
POST	<code>/properties/:propertyId/rooms</code>	OWNER
GET	<code>/properties/:propertyId/rooms</code>	public
GET	<code>/rooms/:roomId</code>	public
PATCH	<code>/rooms/:roomId</code>	OWNER
DELETE	<code>/rooms/:roomId</code>	OWNER soft delete

Seat safety: `Availability.seatsLeft` changes only via atomic `updateMany({ seatsLeft: { gt: 0 } })` in a `$transaction`.

6. Roommate Matching — `/roommates`

Method	Endpoint	Auth
PUT	<code>/roommates/profile</code>	TENANT upsert
GET	<code>/roommates/profile</code>	TENANT
GET	<code>/roommates/matches</code>	TENANT scored matches
GET	<code>/roommates/room-matches</code>	TENANT room suggestions

Scoring: budget overlap + location + lifestyle-tag overlap.

7. Viewing Requests — `/viewing-requests`

Method	Endpoint	Auth
POST	<code>/viewing-requests</code>	TENANT { <code>propertyId</code> , <code>roomId</code> , <code>requestedDate</code> , <code>note?</code> }
GET	<code>/viewing-requests/my</code>	TENANT
GET	<code>/viewing-requests/property/:propertyId</code>	OWNER
PATCH	<code>/viewing-requests/:id/status</code>	OWNER { <code>status</code> : <code>PENDING</code> <code>APPROVED</code> <code>REJECTED</code> <code>COMPLETED</code> }

8. Applications — `/applications`

Method	Endpoint	Auth
POST	<code>/applications</code>	TENANT apply { <code>roomId</code> , <code>message?</code> }
GET	<code>/applications/my</code>	TENANT
GET	<code>/applications/property/:propertyId</code>	OWNER
PATCH	<code>/applications/:applicationId/review</code>	OWNER { <code>status</code> : <code>APPROVED</code> <code>REJECTED</code> }

Approve = one `$transaction`: reserve seat + mark APPROVED + auto-reject other pendings + create Lease.

9. Leases — /leases

Method	Endpoint	Auth
GET	/leases/my	TENANT
GET	/leases/property/:propertyId	OWNER
GET	/leases/:leaseId	scoped
PATCH	/leases/:leaseId/terminate	OWNER releases seat

10. Rent Payments — /rent-payments

Method	Endpoint	Auth
POST	/rent-payments/lease/:leaseId/generate	OWNER { months }, idempotent
GET	/rent-payments/lease/:leaseId	scoped list by dueDate
POST	/rent-payments/:id/pay	TENANT initiates SSLCommerz, returns { gatewayUrl }

Overdue flagging via `markOverdueRents()` (callable, no live cron wired).

11. Utility Bills — /utility-bills

Method	Endpoint	Auth
POST	/utility-bills	OWNER create + auto-split, remainder to first tenant
GET	/utility-bills/property/:propertyId	OWNER
GET	/utility-bills/my-splits	TENANT
POST	/utility-bills/splits/:id/pay	TENANT via SSLCommerz

12. Maintenance — /maintenance

Method	Endpoint	Auth
POST	/maintenance	TENANT { roomId, title, description, priority? }
GET	/maintenance/my	TENANT
GET	/maintenance/property/:propertyId	OWNER
PATCH	/maintenance/:id/status	OWNER { status: OPEN IN_PROGRESS RESOLVED }

13. Documents — /documents

Method	Endpoint	Auth
POST	/documents	Bearer multipart { file, type, leaseId? } to Cloudinary
GET	/documents/my	Bearer
GET	/documents/:id	scoped
DELETE	/documents/:id	owner

Allowed: jpeg, png, webp, pdf. Max 5 MB.

14. Notifications — /notifications

Method	Endpoint	Auth
GET	/notifications	Bearer
PATCH	/notifications/:id/read	Bearer
PATCH	/notifications/read-all	Bearer

Auto-fired on: application review, viewing status, maintenance status, payment success, lease termination.

15. Payments (SSLCommerz) — /payments

Method	Endpoint	Auth
GET	/payments/my	Bearer
GET	/payments/by-tran/:tranId	Bearer
GET	/payments/success? tran_id=..&val_id=..	callback, re-validates via val_id then redirects to frontend
GET	/payments/fail?tran_id=..	callback
GET	/payments/cancel?tran_id=..	callback
POST	/payments/ipn	server-to-server source of truth

Rows start INITIATED, only SUCCESS after server-side validation.

16. Dashboard — /dashboard

Method	Endpoint	Auth
GET	/dashboard/owner	OWNER stats
GET	/dashboard/admin	ADMIN stats

17. Assistant — /assistant (not in Postman collection)

Method	Endpoint	Auth
POST	/assistant/chat	public { history: [{ role: user assistant, content }]} }

Needs GEMINI_API_KEY on backend, else AI assistant is not configured.

Quick test flow

1. Register/login as owner, create property, create room.
2. Register/login as tenant, create viewing request, apply to room.
3. As owner approve application, lease is created.
4. Generate rent schedule, pay rent via SSLCommerz sandbox.
5. Raise maintenance request, upload document, check notifications.